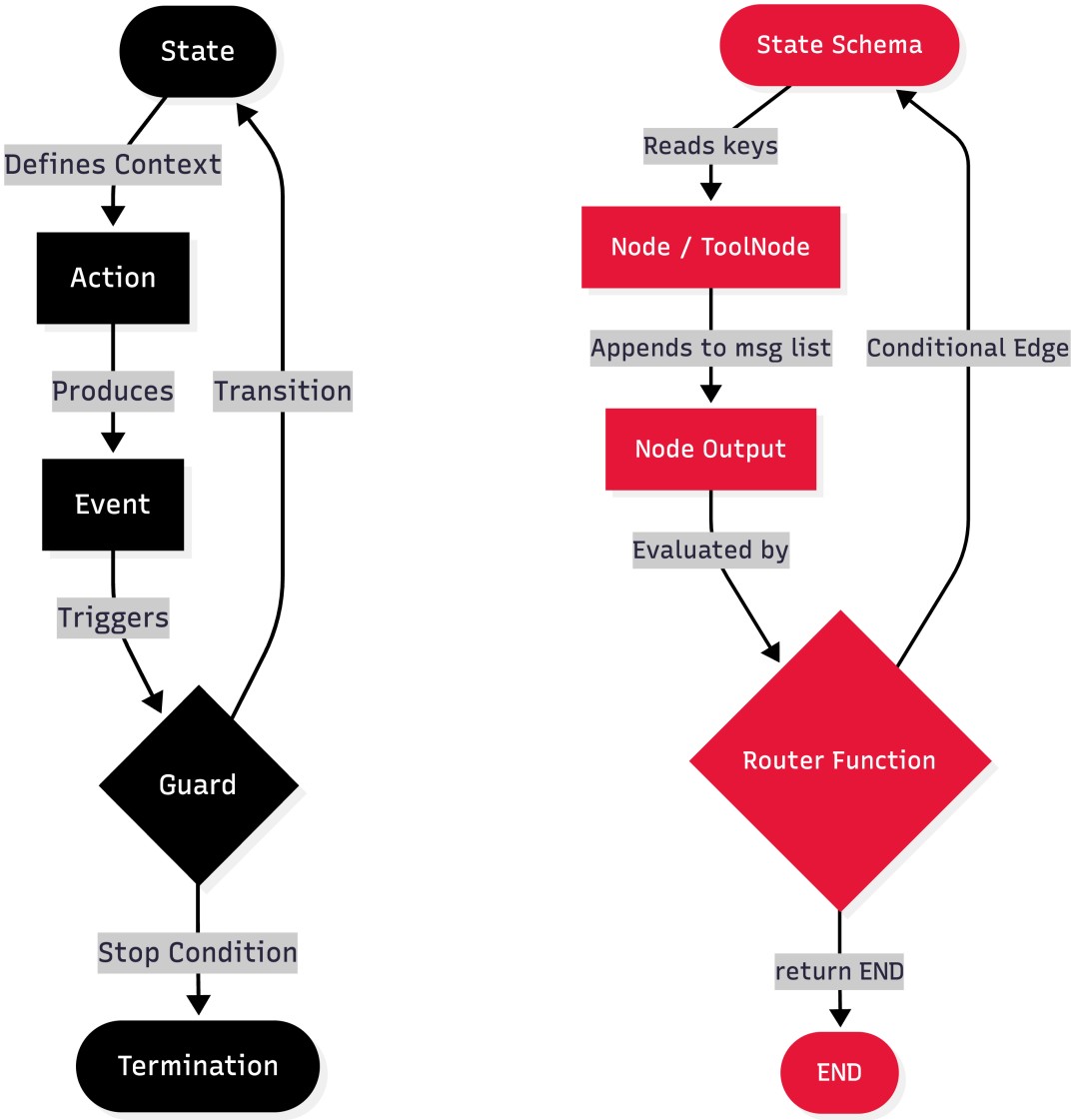
Cheat Sheet: State Machines mapping to LangGraph

A compact reference for translating finite state machines and hierarchical state machines into LangGraph graphs, subgraphs, routers, and checkpointed threads.



(a) Finite State Machine (b) LangGraph execution flow

Figure 1: FSM vs. LangGraph flow.

Core vocabulary mapping: FSM to LangGraph

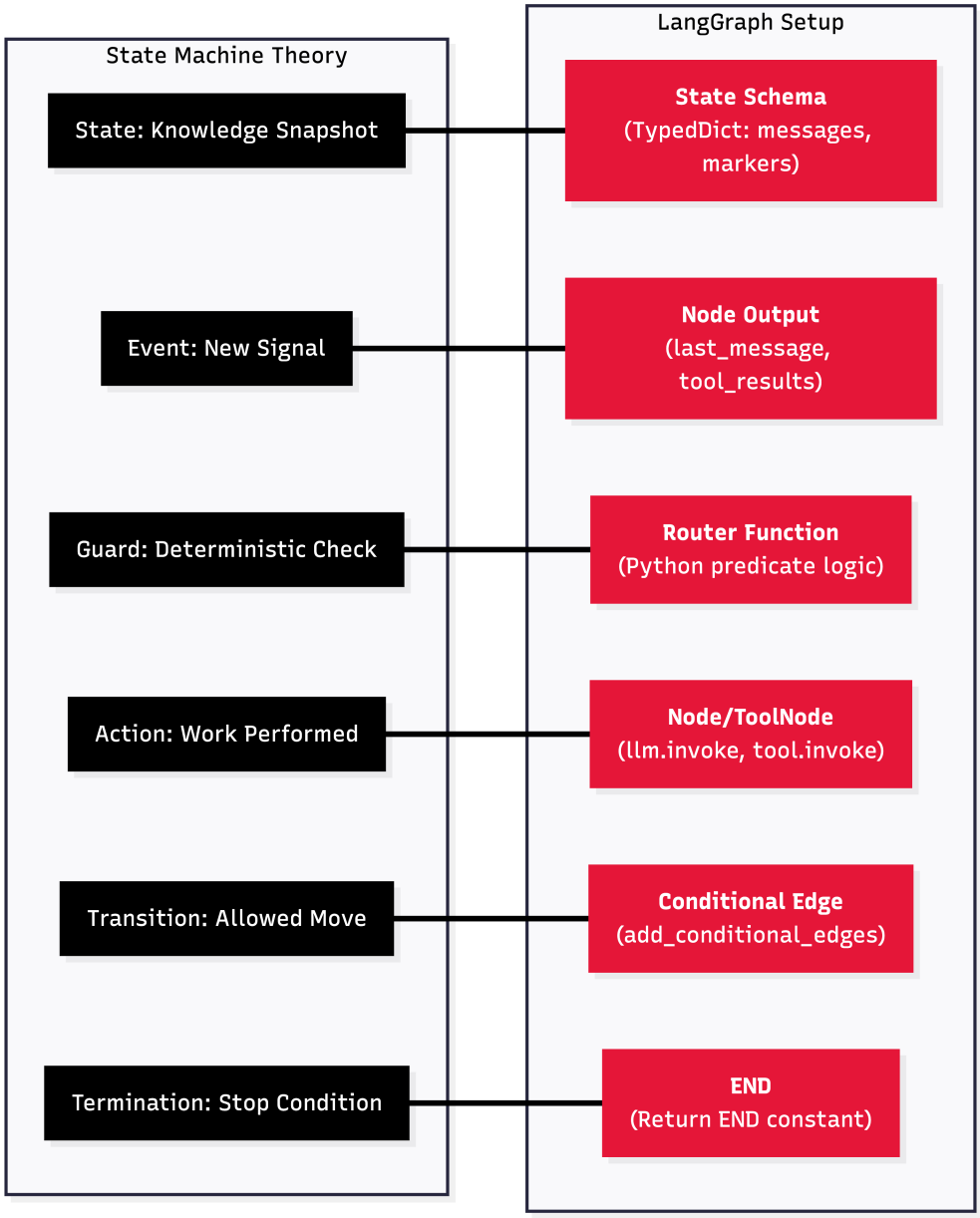


Figure 2: Visualized concept mapping FSM to LangGraph.

State machine concept	Meaning in agents	LangGraph primitive	What you implement
State	Snapshot of knowledge and progress	Typed state schema	Define shared keys such as messages, routing hints, progress markers
Event	New signal since last decision	Last message, tool output, node output	Read last message, tool calls, tool results, user input
Guard	Deterministic check that selects a transition	Router used by conditional edges	Pure Python predicate returning a route key
Action	Work performed during a transition	Node function or ToolNode	Call model, call tools, update state, request human input
Transition	Allowed move between states	Edge or conditional edge	Explicit edges plus a route map
Termination	Stop condition	END	Return END in router or route mapping

Example: Typed state schema + shallow history marker

Maps the notion of state to a LangGraph state schema. The `working_last` key acts as a shallow history marker when re-entering a superstate.

```
from typing import List, TypedDict
from langchain_core.messages import BaseMessage

class State(TypedDict):
    messages: List[BaseMessage]
    working_last: str | None
```

FSM in LangGraph

A finite state machine is a flat graph with a small set of nodes and explicit transitions. Each node is a stable mode and each conditional edge is a guarded transition.

- State schema defines what persists between steps
- Nodes implement step logic and update the state
- Routers inspect state and decide the next node
- END represents termination
- Useful when recoverability matters, checkpoint after each node

Example: Tool-calling FSM loop

A canonical LangGraph FSM: the LLM node reasons, ToolNode executes tool calls, and a router decides whether to continue or finish based on `tool_calls`.

```
from typing import Annotated, List, TypedDict
from langchain_core.messages import BaseMessage
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode

class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]

def llm_node(state: AgentState) -> AgentState:
    ai = llm.invoke(state["messages"])
    return {"messages": [ai]}

tool_node = ToolNode(tools=tools)

graph = StateGraph(AgentState)
graph.add_node("llm", llm_node)
graph.add_node("tools", tool_node)
graph.add_edge(START, "llm")

def route(state: AgentState):
    last = state["messages"][-1]
    calls = getattr(last, "tool_calls", None) or []
    return "tools" if calls else END

graph.add_conditional_edges("llm", route, {"tools": "tools", END: END})
graph.add_edge("tools", "llm")
```

HSM in LangGraph

A hierarchical state machine is a parent graph plus one or more subgraphs. A compiled subgraph behaves like a superstate node whose internal nodes are substates.

- Superstate equals a compiled subgraph used as a node in the parent graph
- Substates equal internal nodes inside the subgraph
- Shared superstate interface equals overlapping state keys read and written by parent and child
- Entry and exit correspond to the subgraph START and the parent edge returning from the subgraph
- Attach shared policy at the parent level, all substates inherit it

Example: Compile a subgraph as a superstate

Defines PLAN/ACT substates, compiles them into a subgraph, and uses the compiled subgraph as a superstate node in the parent graph.

```
from langchain_core.messages import AIMessage
from langgraph.graph import StateGraph, START

def plan_node(state: State) -> State:
    ai = AIMessage(content="Plan next step")
    return {"messages": [ai], "working_last": "plan"}

def act_node(state: State) -> State:
    ai = AIMessage(content="Act on plan")
    return {"messages": [ai], "working_last": "act"}

subgraph_builder = StateGraph(State)
subgraph_builder.add_node("PLAN", plan_node)
subgraph_builder.add_node("ACT", act_node)
subgraph_builder.add_edge(START, "PLAN")
subgraph = subgraph_builder.compile()
```

Guards and routing

Where the guard lives	What it controls	Typical examples
Inside a subgraph	Routing among substates	Plan to act, act to reflect, retries, tool loop
At the parent level	Global policy across a superstate	Budget caps, safety gates, stop signals, rate limits

Example: Guards via routers (child + parent)

Implements deterministic routing. A child router chooses the next internal substate. A parent-level guard enforces global policy before continuing or ending.

```
from langgraph.graph import END
from langgraph.graph import StateGraph, START

def route_within_working(state: State):
    last = state.get("working_last")
    return "ACT" if last == "plan" else END

subgraph_builder.add_conditional_edges(
    "PLAN",
    route_within_working,
    {"ACT": "ACT", END: END},
)

parent = StateGraph(State)
parent.add_node("WORKING", subgraph)

def global_guard(state: State):
    tail = "".join(getattr(m, "content", "") for m in state["messages"][-3:]).lower()
    return END if "stop" in tail else "WORKING"

parent.add_edge(START, "WORKING")
parent.add_conditional_edges("WORKING", global_guard, {"WORKING": "WORKING", END: END})
```

History and checkpointing

Checkpointing restores the last completed node for a given thread. For explicit resume semantics, persist a small marker such as `working_last` and branch on re-entry.

- MemorySaver provides per-thread checkpoints
- `thread_id` isolates branches and enables parallel conversations
- Explicit marker such as `working_last` can emulate shallow history inside a superstate

Example: Checkpointing with thread IDs

Adds thread-based persistence. `MemorySaver` checkpoints state so a thread can resume from the last completed node.

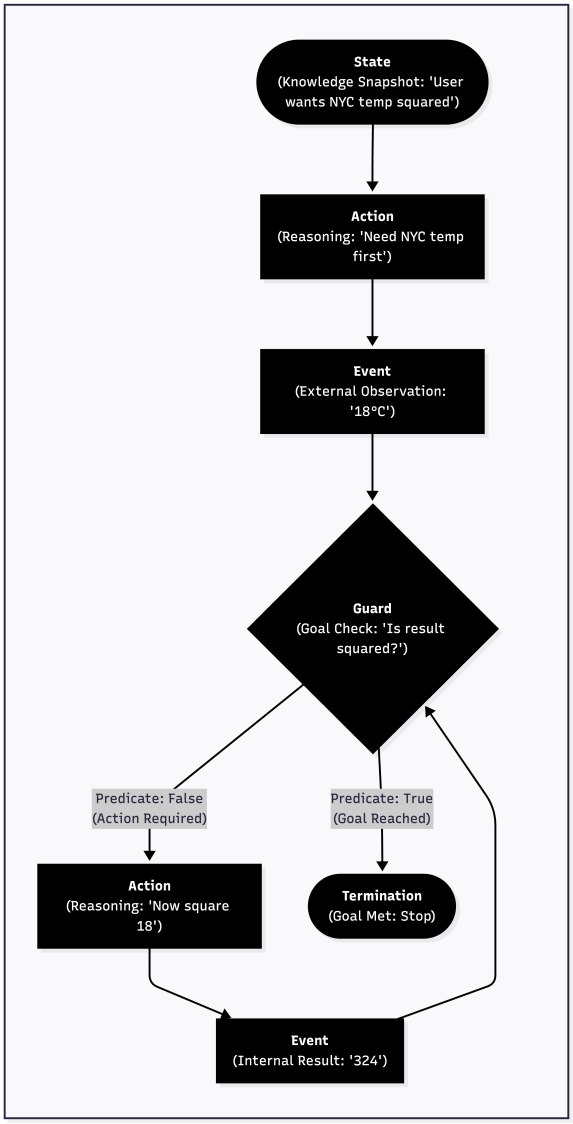
```
from langchain_core.messages import HumanMessage
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
app = parent.compile(checkpointer=checkpointer)

cfg = {"configurable": {"thread_id": "session-1"}}
app.invoke({"messages": [HumanMessage(content="start")], "working_last": None},
↪  config=cfg)
```

Canonical mapping patterns

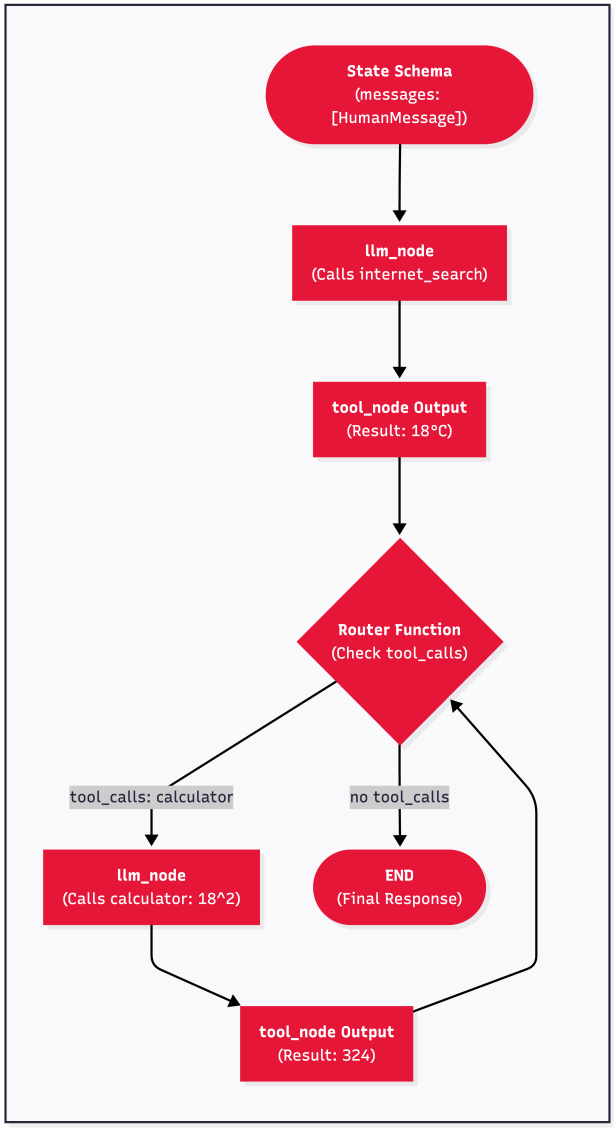
Pattern	Graph shape	Key router rule
Tool calling FSM	START to llm, llm to tools or END, tools to llm	If last message has tool calls route to tools else END
Plan act reflect HSM	Parent node WORKING is a subgraph with PLAN, ACT, REFLECT	Branch inside WORKING based on `working_last` or evaluation outputs
Supervisor with specialists	Parent supervisor routes into multiple subgraphs	Global guard enforces budget and safety before dispatch

The State Machine in Action: Weather Example

This example demonstrates how a simple task (get temperature → square it) maps to a continuous reasoning–acting–adapting cycle. It highlights how **State** persists across turns using **Checkpoints**.



(a) FSM Blueprint: The logical loop of Reasoning (Action) and Observation (Event).



(b) LangGraph Implementation: The physical flow through Nodes, ToolNodes, and the Router.

Figure 3: FSM vs. LangGraph flow.

FSM Phase	Execution Detail
Action 1	LLM decides it lacks current data and calls `internet_search`.
Event 1	The search tool returns **18°C**, updating the shared State.
Guard check	The Router sees tool results but notes the "Square" requirement is unmet.
Action 2	LLM retrieves the 18°C from History and calls the `calculator` tool.
Termination	Router sees a final answer and transitions the system to **END**.

Example: Multi-turn stateful execution

This logic executes two distinct turns. Turn 1 populates the state with the temperature; Turn 2 restores that state via `thread_id` to perform the calculation.

```
# Setup the State and Graph (as defined in FSM section)
cfg = {"configurable": {"thread_id": "nyc-session-42"}}

# --- TURN 1: Reasoning -> Search Action -> State Update ---
# Input: "Get NYC temperature in Celsius"
turn1_input = {"messages": [HumanMessage(content="Get NYC temperature in Celsius.")]}
app.invoke(turn1_input, config=cfg)
# Result: State now contains [HumanMessage, AIMessage(tool_call), ToolMessage(18C)]

# --- TURN 2: Restore History -> Reasoning -> Calculate Action ---
# Input: "Now compute the square of that"
# Note: We do NOT need to repeat the temperature; the Guard/History restores it.
turn2_input = {"messages": [HumanMessage(content="Now compute the square of that.")]}
final_output = app.invoke(turn2_input, config=cfg)

# The FSM 'Guard' (Router) ensures the flow stays in the loop until END
print(final_output["messages"][-1].content) # Output: "The square of 18 is 324."
```

Execution Phase	State Machine Mapping
User Prompt	**Event:** The trigger that initiates the first state transition.
LLM reasoning	**State Transition:** Moving from START to the 'Reasoning' mode.
Tool invocation	**Action:** Performing work (Searching/Calculating) to update the State.
Router check	**Guard:** Checking if `tool_calls` exists to determine the next Transition.
Thread persistence	**Deep History:** Using the checkpoint to resume the 'Circle' at Turn 2.

Multi-Agent Orchestration (HSM)

In this Hierarchical State Machine (HSM), the Supervisor node acts as a Superstate managing specialized Sub-graphs. This pattern is ideal for scaling complex workflows.

HSM Concept	LangGraph Realization
Superstate	A **Compiled Subgraph** used as a node in the parent graph.
Superstate Policy	Global guards (e.g., total MAS budget) defined in the parent router.
Substate Interface	Overlapping keys in the `TypedDict` (the shared Bus).
Shallow History	Using `working_last` markers to resume a specific subgraph node.

In figures 4 and 5, the *Supervisor* acts as the *governing FSM*, while the specialized agents function as internal *sub-circles*. This mirrors the book's analogy of moving from a *Lone Contributor* to a *Tech Lead* (the Supervisor) managing *Specialized Teams* (Sub-graphs).

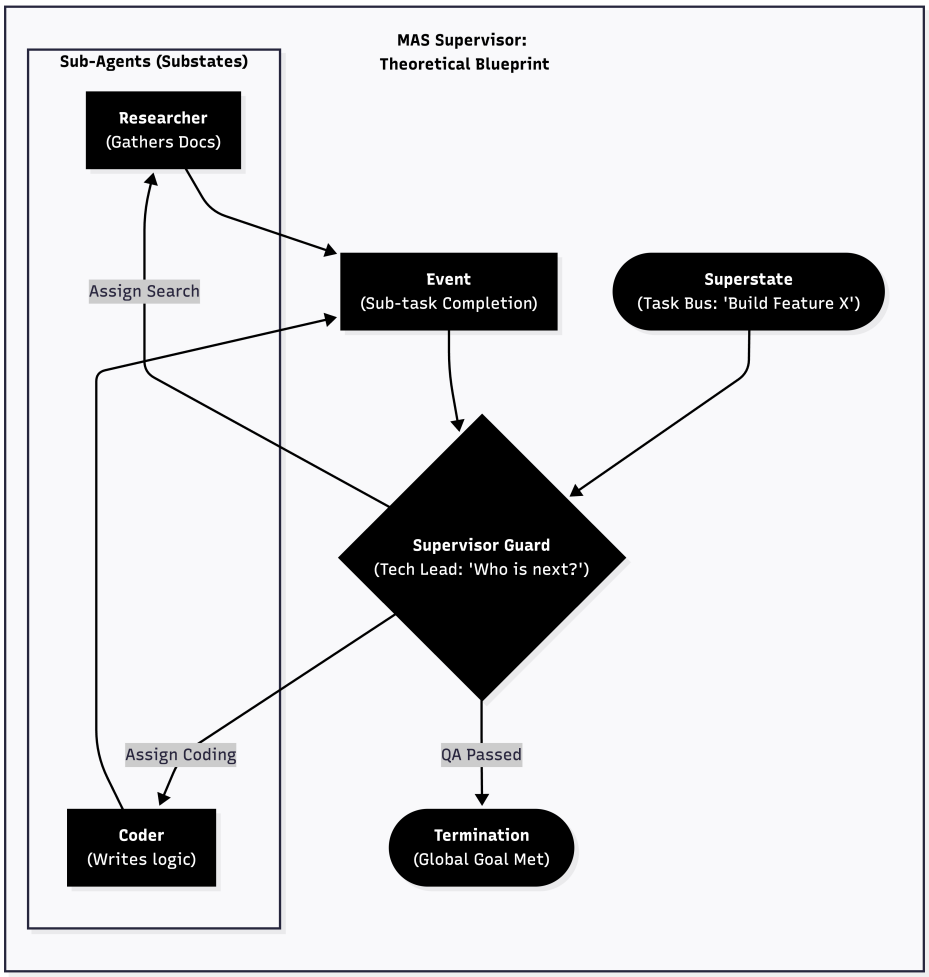


Figure 4: Supervisor theory: the 'tech lead' logic.

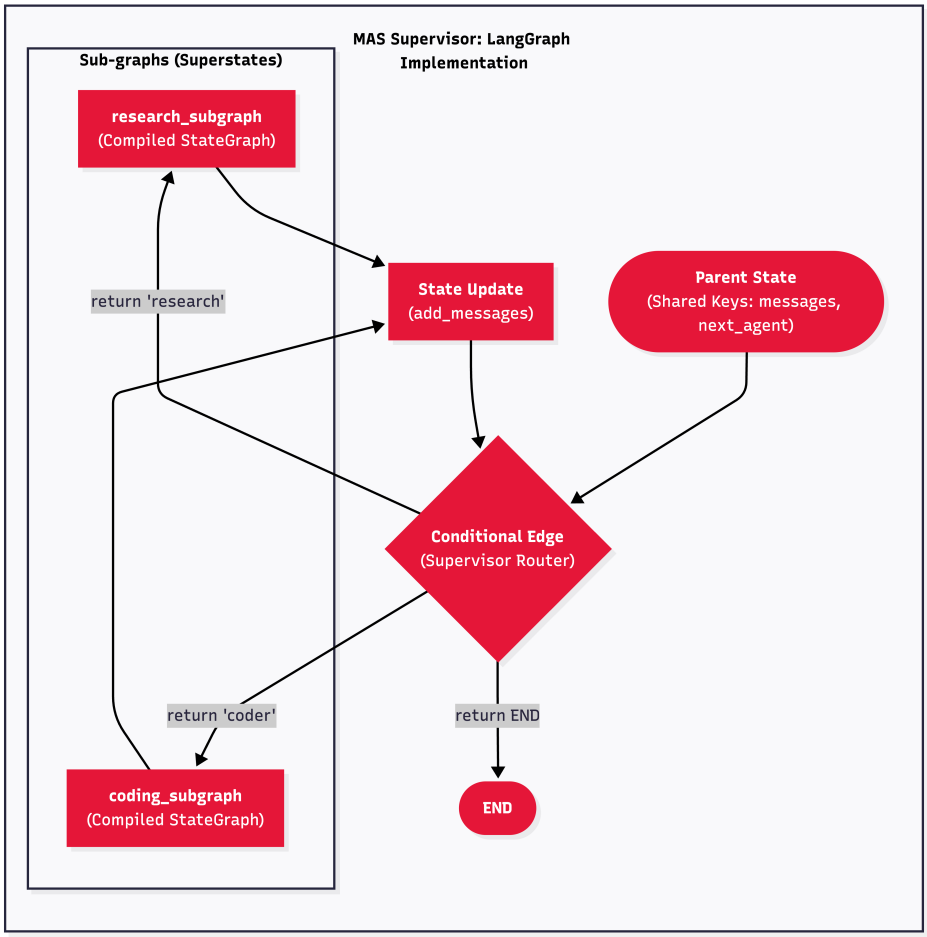


Figure 5: Supervisor code: subgraphs as superstates.

Build checklist

Define state keys, add nodes for stable modes, encode transitions with edges and conditional edges, keep guards deterministic, compile with a checkpointer, and assign thread IDs for persistence and branching.